AIO High Achievers Camp · Session 1: Prefix Sum

演讲者备注（Speaker Notes）

每页的备注与本次课 slides 中的 speaker notes 相同；课堂练习的答案都写在对应页的备注里。页码与 AIO High Achievers Camp Session 1 Prefix Sum.pptx 一致。

第 1 页 · Prefix Sum

这一系列共 4 次课，每次约 2 小时。贯穿全部课程的主线是 AMT 的四大核心技巧：Prefix Sum、Binary Search、Two Pointers 和 Greedy。 第 1 次课讲 Prefix Sum：先讲普通的 prefix sum，再讲"一轮的净变化"（net change per round），最后讲二阶差分（second-order differences）。

第 2 页 · AMT's four core techniques

四种颜色在 4 次课里保持不变：青色 = Prefix Sum，橙色 = Binary Search，珊瑚红 = Two Pointers，深蓝 = Greedy。每道题开始时都先标出它用到哪几个技巧。

第 3 页 · Three questions

强调：P6 很少是"直接套模板"。四大技巧只是骨架，真正的难点是找到 key observation，把题目化简到能套模板为止。右侧空白用来写学生的回答。 参考答案（以今天的题为例）：

1. Brute force：Spaceship 枚举每条边的流量；Tennis 直接模拟（可达 2×10^11 步）；Twin Rivers 枚举新桥位置 p 再重算所有行程，O(L·T)。
2. 单调的量：Tennis 中每个箱子的球数随轮数线性变化；Twin Rivers 中桥的位置已排序，可以 Binary Search 找最近的桥。
3. 可累加 / 局部决定的量：Spaceship 的边流量是 prefix sum；Tennis 的一轮净变化是 prefix sum；Twin Rivers 的总代价可用二阶差分累加。

第 4 页 · One technique per session

每道 Problem 5 都和同一次课的一道 Problem 6 配对：核心想法相同，化简步骤更短，用来巩固。

第 5 页 · Spaceship Shuffle

约 30 分钟。讲解路线：把环剪成链（纯 prefix sum）→ 回到环（多一个变量 f）→ 中位数（median）。

第 6 页 · The problem

提问：为什么只看净流量（net flow）？ 答案：如果同一条边上有人往左、有人往右，让他们互换身份，花费只会减少。所以最优解里每条边只有一个方向。 右侧空白可以画一个 6 个 cabin 的环。

第 7 页 · Cut the ring into a chain

在黑板上画链 5 2 1 6 5 1 → 3 3 2 7 3 2，从左往右逐条边算 P[i]，让学生看到"左边多出来的人必须穿过这条边"。 参考：A – B = 2 –1 –1 –1 2 –1，P = 2 1 0 –1 1 0，链上答案 ΣIP[i]I = 2+1+0+1+1+0 = 5。

第 8 页 · Back to the ring: one extra variable

提问：为什么 flow[N] = f 也符合公式 P[N] + f？ 答案：因为 ΣA = ΣB，所以 P[N] = 0。

第 9 页 · Stand at x, step one to the right

在下方空白画数轴，标出 P = –1 0 0 1 1 2。让学生站在不同位置，算到所有点的距离之和。 参考答案：站在 –1：0+1+1+2+2+3 = 9；站在 0：1+0+0+1+1+2 = 5；站在 1：2+1+1+0+0+1 = 5；站在 2：3+2+2+1+1+0 = 9。 最小值 5 出现在 0 和 1 之间（两个中位数之间都可以）。这就是 Greedy：一直往更好的方向走，直到走不动为止。

第 10 页 · Sample 1

答案：A – B = 2 –1 –1 –1 2 –1；P = 2 1 0 –1 1 0；排序后 –1 0 0 1 1 2；median 取 1（取 0 也可以）；答案 ΣIP – medI = 5。 再看样例 2：A = 5 0 0 0 0，B = 1 1 1 1 1，P = 4 3 2 1 0，median = 2，答案 6。如果只能单方向走（f = 0），答案是 10：这就是"环"带来的好处。

第 11 页 · Prefix sums, then sort for the median

强调 long long：P 最大可到 10^5 × 10^5 = 10^10。复杂度 O(N log N)，瓶颈是 sort。

第 12 页 · Subtasks and common mistakes

挑战题：如果第 i 条边每人收 w[i] 元，中位数还成立吗？ 答案：不能用普通中位数，要用带权中位数（weighted median）：把每个 P[i] 看成权重为 w[i] 的点，取使左右权重平衡的点。

第 13 页 · Tennis Robot II

约 25 分钟。Spaceship 的循环在空间上（cabin 围成一圈），这题的循环在时间上（指令一轮一轮重复）。同一个技巧：把一整轮压缩成一个摘要。

第 14 页 · The problem

问"三问"中的第 1 问：brute force 是什么？为什么慢？ 答案：直接模拟；最多约 10^6 轮 × 2×10^5 条指令 = 2×10^11 步。慢在"轮数太多"，接下来就要把轮数压缩掉。

第 15 页 · Summarise one round

特殊情况：s < 1 表示箱子在第 1 轮就出问题（t = 0）；d[i] ≥ 0 表示箱子永远不会出问题（跳过）；如果没有任何箱子会出问题，输出 FOREVER。 为什么不用 Binary Search？每个箱子的条件都是关于 r 的一次不等式，可以直接解出来。对 R 二分也可以（条件是单调的），但除法更短，边界情况更少。

第 16 页 · Same idea: compress the loop

先让学生口头比较两题，再显示表格。 参考答案就是表格内容：循环分别在时间上 / 空间上；Prefix Sum 分别求最低点 mn[i] 和净变化 d[i] / 每条边的流量 P[i]；之后分别用除法算轮数再模拟一轮 / 取中位数。

第 17 页 · Sample 3

答案： 箱子 1：指令 1 前变化 0 → mn = 0，d = −1，s = 3 → t = (3 − 1) / 1 + 1 = 3。 箱子 2：指令 3 前变化 0 → mn = 0，d = 0，s = 2 → 永远不会出问题。 箱子 3：没被取过，d = +2。 箱子 4：指令 2 前变化 0，指令 4 前变化 −1 → mn = −1，d = −2，s = 3 → t = (3 − 1) / 2 + 1 = 2。 箱子 5：没被取过，d = +1。 所以 R = min(3, 2) = 2。第 3 轮开始时箱子 1 有 1 个、箱子 4 有 0 个。指令 1（1→5）成功；指令 2 要从箱子 4 取球，空了，停下。答案 = 2 × 4 + 1 = 9。 手动验证箱子 4：第 2 轮最低点 3 − 2 = 1（可以），第 3 轮最低点 3 − 4 = −1（失败）。

第 18 页 · Rounds by division

答案最大可到 10^6 × 2×10^5 = 2×10^11，必须用 long long。整数除法向下取整，正好是公式需要的 floor。 之后：b[i] = x[i] + R·d[i]，从这里模拟一轮，答案 = R·M + 这一轮成功的条数。完整程序见 2024_P5_Tennis_Robot_II.cpp。

第 19 页 · Atlantis III: Twin Rivers

约 30 分钟。目标：学生要记住"差分数组 + 一次 prefix sum"能还原分段常数函数（step function），"二阶差分 + 两次 prefix sum"能还原分段线性函数（piecewise-linear function）。

第 20 页 · The problem

在右侧画出三条陆地和两条河，用样例 1：River 1 的桥在 2 和 6，River 2 的桥在 4。

第 21 页 · How long is one trip?

Brute force：枚举每个 p，再重算所有行程，O(L·T) ≈ 10^11，只能过 subtask 1。 参考：去 Strip 3 的行程 X = 4，经桥 2 和 4：2 + 2·(4 − 2) = 6；经桥 6 和 4 也是 6。

第 22 页 · Each trip's cost is a simple shape in p

两种行程的代价函数都是折线： 去 Strip 2：V 形 1 + 2|X − p|，在 old 处被截平（新桥离 X 太远就不如旧桥）。 去 Strip 3：trough（槽形）：在 [u2, X]（或 [X, w2]）上是平底，离开平底每 1 km 增加 2，同样被 old 截平。平底高度是 2 + 2·(X − u2)。 红点是拐点（转折点）：斜率只在这些地方改变（−2、0、+2 之间切换）。每个行程最多只有几个拐点，这就是后面能用二阶差分的关键。

具体例子（样例 1，新桥建在 River 1）：行程 X = 4 去 Strip 3：u2 = 4，平底只有一个点，w = 6 6 4 2 4 6 6；行程 X = 3 去 Strip 3：只有右边的 w2 = 4，平底是 [3, 4]、高度 4，w = 6 6 4 4 6 6 6（p = 1..7）。

第 23 页 · Why not just try every p?

Brute force：L 约 10^6 个位置 p，每个 p 都要把 2×10^5 个行程的代价加起来，共约 2×10^11 次运算，远远超时（1 秒大约只能做 10^8 次）。 关键：每个行程的 w(p) 是一条折线，只有少数几个拐点（转折点）。我们只记下拐点处的信息（每个行程 O(1) 个位置），最后用两次 prefix sum 一次性还原出所有 p 上的总代价（O(L)）。再加上每个行程用 Binary Search 找最近的桥 O(log N)，总复杂度 O(T log N + L)，约 2×10^5 × 18 + 10^6 ≈ 5×10^6。 提问：能不能每个行程只算几个 p？答案：我们要的是"所有行程代价之和"在每个 p 上的值，最小值可能出现在任何 p，所以每个 p 的总和都要知道——但不需要一个 p 一个 p 地去加。 为了做到这一点，需要两个工具：差分数组（下一页）和二阶差分（再下一页）。

第 24 页 · Store the jumps, not the values

差分（difference）就是相邻两项的差：D[i] = a[i] − a[i−1]。它记录的是"在哪里、跳了多少"。 答案：D = 3 0 0 4 0 0 −5 0；D 的前缀和 = 3 3 3 7 7 7 2 2 = a。差分和 prefix sum 互为逆运算。 经典用法（区间加）：把 a[2..4] 都加 5 → 只改 D[2] += 5、D[5] −= 5，最后做一次前缀和得到 3 8 8 12 7 7 2 2。每次修改 O(1)，最后还原 O(n)。 局限：差分数组只在"阶梯形"（分段常数，step function）的数据上省事。如果数据是斜线（每一格都在变），D 处处不为 0，就没有节省了——这就是下一页要讲二阶差分的原因。

第 25 页 · Store the bends, not the values

答案：斜率（一阶差分）w(p) − w(p−1)，p = 2..7：0, −2, −2, 2, 2, 0。 d2（斜率的变化），p = 2..6：−2, 0, 4, 0, −2。只有在拐点 p = 2、4、6 处不为 0。 为什么？折线在两个拐点之间是直线，斜率不变，所以斜率的差是 0；只有在拐点处斜率才改变。一阶差分（斜率）在斜线上处处不为 0，二阶差分只在拐点不为 0，所以每个行程只需要写 O(1) 个位置。 还原（两次 prefix sum）：已知 w(0) = 6、w(1) = 6，所以 slope(1) = 0。slope 依次加上 d2：0, −2, −2, 2, 2, 0；w 依次加上 slope：6, 4, 2, 4, 6, 6。和原来的 w 完全一样。 代码里的 tot[q+1] = 2·tot[q] − tot[q−1] + d2[q] 就是把这两次 prefix sum 合并成一行：tot[q+1] − tot[q] = (tot[q] − tot[q−1]) + d2[q]，即"新斜率 = 旧斜率 + 斜率变化"。

第 26 页 · Store changes, rebuild with prefix sums

多个行程怎么办？二阶差分是线性的：总代价的 d2 = 每个行程的 d2 相加。所以所有行程都往同一个 d2 数组里写，最后只还原一次。 重点讲还原公式：tot[q+1] = 2·tot[q] − tot[q−1] + d2[q]。把它移项：tot[q+1] − tot[q] = (tot[q] − tot[q−1]) + d2[q]，左边是 q 到 q+1 的斜率，括号里是上一段的斜率，d2[q] 是斜率的变化。所以这一行其实就是两次 prefix sum：先累加 d2 得到斜率，再累加斜率得到 tot。需要两个起点 tot[0] 和 tot[1]（代码里的 v0、v1）。 用样例 1 的行程验证：w(0) = 6、w(1) = 6，d2[1..6] = 0, −2, 0, 4, 0, −2 → w(2) = 2·6 − 6 + 0 = 6，w(3) = 2·6 − 6 − 2 = 4，w(4) = 2·4 − 6 + 0 = 2，w(5) = 2·2 − 4 + 4 = 4，w(6) = 2·4 − 2 + 0 = 6，w(7) = 2·6 − 4 − 2 = 6。

复杂度从 O(T·L) 降到 O(T log N + L)。 提问：普通差分数组（一次 prefix sum）能还原什么样的函数？答案：只能还原分段常数函数（step function）。分段线性函数（折线）需要二阶差分 + 两次 prefix sum。

第 27 页 · Trip X = 4 to Strip 3 (sample 1)

答案：原代价 Old cost：经桥 2 和 4 → 2 + 2·2 = 6（经桥 6 和 4 也是 6）。 w(p)，p = 1..7：6 6 4 2 4 6 6。 d2[p]，p = 2..6：–2, 0, 4, 0, –2（只有转折点 2、4、6 处不为 0）。 整个样例在 River 1 的 p = 4 建新桥：四个行程的代价是 4、2、4、3 → 总和 13。

第 28 页 · The big picture before the code

先讲整体流程，再看代码。程序分 5 步： ① 读入，把桥按河分开存（每条河的桥排好序）。 ② 对每个行程：用 Binary Search 找 X 左右最近的桥（pred / succ）；写出这个行程的新代价 w(p)，分两种情况：f 表示新桥建在 River 1，g 表示建在 River 2；再用 addFunc 把它加进总和——只在转折点附近写 d2，所以每个行程只写 O(1) 个位置。 ③ build：两次 prefix sum，还原出每个 p 的总代价 tot[w][p]。 ④ 扫一遍所有 p，两条河都看，取最小值。 ⑤ 输出答案。 提问：循环里每个行程花多少时间？答案：两次二分 O(log N)，加上 O(1) 个转折点。总复杂度 O(T log N + L)。 提问：④ 里为什么有时不能把新桥建在 River 1？答案：如果有行程要去 Strip 3，而 River 2 上一座桥都没有，这些行程原本就完成不了，新桥必须建在 River 2（样例 3 就是这种情况）。

第 29 页 · Five steps, three helpers

伪代码和完整程序 2024_P6_Twin_Rivers.cpp 一一对应：pred / succ 是两个二分函数；trough(a..b) 表示在 [a, b] 上是平底、两边斜率 ±2 的槽形函数，平底高度是 2 + 2·(b – a)；addFunc 只在转折点附近写 d2；build 做两次 prefix sum。 用样例 1 的行程（X = 4，去 Strip 3）验证：u2 = w2 = 4，所以 f 的两个 trough 都是 [4, 4]、高度 2；old = 6。f(p) = 6 6 4 2 4 6 6（p = 1..7），和手算页的结果一致。 去 Strip 2 的行程：新桥建在 River 2 对它没有帮助，所以 g = old（常数函数，d2 全是 0）。

第 30 页 · Write d2, then two prefix sums

转折点可能落在半整数上（两段函数相交处），所以代码在每个转折点附近多检查几个整数（用 2 倍坐标存）。完整程序：2024_P6_Twin_Rivers.cpp。复杂度 O(T log N + L)。

第 31 页 · Write Spaceship Shuffle from scratch

下去巡视。最常见的 bug：用 int 而不是 long long；q 没有从下标 1 开始排序；把 p 原地排序后又拿来用（应该排序一份拷贝）。 样例答案：样例 1 = 5，样例 2 = 6，样例 3 = 3。

第 32 页 · Three faces of Prefix Sum

让每个学生说出今天的一道题，以及它用了哪个技巧。 参考答案：Spaceship Shuffle：Prefix Sum（每条边的流量）+ Greedy（中位数）；Tennis Robot II：Prefix Sum（一轮的最低点和净变化）+ 除法算轮数；Twin Rivers：二阶差分 + 两次 Prefix Sum，再用 Binary Search 找最近的桥。

第 33 页 · Homework

下次课：Binary Search 和 Two Pointers。提醒学生把课程笔记里的 binary search 模板带来。